Design and Analysis of Algorithms 12/01/2026

Lecture 1: Perceptron Learning and GCD algorithm

Instructor: Sourav Chakraborty *Scribe: Shreyas Gangopadhyay, Vineet Gaurav*

1 Algorithm Design

There are 3 steps for designing an algorithm.

1. Understanding the actual problem.
2. Designing the actual algorithm—a step-by-step instruction to solve the problem.
3. Checking the properties of the algorithm, like correctness, time complexity, space complexity, etc.

2 Two Algorithms

2.1 Perceptron Learning Algorithm

The Perceptron Learning algorithm aims to solve the problem of uniquely classifying two sets of points. The input to the algorithm is a set of data points (each d-dimensional) and their associated labels ($+$ or $-$). For the ease of mathematical computations, we may replace the $+$ label with the number $+1$, and the $-$ label with the number -1. Hence, the input becomes

$$(\mathbf{x}^1, y^1), (\mathbf{x}^2, y^2), \ldots, (\mathbf{x}^n, y^n),$$

where for $i \in \{1, 2, \ldots, n\}$, $\mathbf{x}^i \in \mathbb{R}^d$ and $y^i \in \{-1, 1\}$.

Problem Statement: Perceptron Binary Classification

Given a training dataset $\mathcal{D} = \{(\mathbf{x}^1, y^1), (\mathbf{x}^2, y^2), \ldots, (\mathbf{x}^n, y^n)\}$, where:

- $\mathbf{x}^i \in \mathbb{R}^{d+1}$ is the input feature vector (including a bias term $x^0 = 1$).

- $y^i \in \{-1, +1\}$ is the class label associated with each $\mathbf{x}^i$.

Objective:

Assuming that the data is linearly separable, the goal is to find a weight vector $\theta \in \mathbb{R}^{d+1}$ such that the linear decision boundary defined by $\theta^T \mathbf{x}$ correctly separates the two classes. In other words, the goal of the algorithm is to find a vector θ that satisfies the following condition for all $i = 1, \ldots, n$:

$$\text{sign}(\theta^T \mathbf{x}_i) = y_i \tag{1}$$

This is equivalent to finding θ such that the functional margin for every point is positive:

$$y_i(\theta^T \mathbf{x}_i) > 0, \quad \forall i \in \{1, \ldots, n\} \tag{2}$$

Note: Frank Rosenblatt, a psychologist at the Cornell Aeronautical Laboratory, developed the Perceptron Learning Algorithm in 1957. He aimed to mimic the working of neurons.

Algorithm 1 Perceptron Learning Algorithm

Initialize $\theta = \mathbf{0}$
while there are misclassified points $(\mathbf{x}^i, y^i)$ **do**
    $\theta \leftarrow \theta + y^i \mathbf{x}^i$
end whilereturn θ

Analysis: Let us assume that the training set is linearly separable with a margin of width γ such that there exists a unit vector θ^* (where $\|\theta^*\| = 1$) satisfying $y_i(\theta^* \cdot \mathbf{x}_i) \geq \gamma$ for all $i \in \{1, \ldots, n\}$. Let $R = \max \|\mathbf{x}_i\|$. We want to find θ such that:

$$y_i(\theta^T \mathbf{x}_i) > 0, \quad \forall i \in \{1, \ldots, n\} \tag{3}$$

It follows from the algorithm that if it terminates, the resulting weight vector is one of the linear separators of the data.

We initialize $\theta_1 = 0$. Let θ_{k+1} be the weight vector after k updates. Suppose $\mathbf{x}_j$ the point is misclassified during iteration k.

1. Lower Bound on $\|\theta_{k+1}\|$

$$\begin{aligned}\theta_{k+1} \cdot \theta^* &= (\theta_k + y_j \mathbf{x}_j) \cdot \theta^* \\ &= \theta_k \cdot \theta^* + y_j(\mathbf{x}_j \cdot \theta^*) \\ &> \theta_k \cdot \theta^* + \gamma\end{aligned}$$

By induction, since $\theta_1 = 0$, it follows that $\theta_{k+1} \cdot \theta^* > k\gamma$. Using the Cauchy-Schwarz inequality, $\theta_{k+1} \cdot \theta^* \leq \|\theta_{k+1}\|\|\theta^*\|$. Since $\|\theta^*\| = 1$:

$$\|\theta_{k+1}\| > k\gamma \tag{4}$$

2. Upper Bound on $\|\theta_{k+1}\|$

$$\begin{aligned}\|\theta_{k+1}\|^2 &= \|\theta_k + y_j \mathbf{x}_j\|^2 \\ &= \|\theta_k\|^2 + \|y_j \mathbf{x}_j\|^2 + 2y_j(\theta_k \cdot \mathbf{x}_j) \\ &\leq \|\theta_k\|^2 + \|\mathbf{x}_j\|^2 + 0 \\ &\leq \|\theta_k\|^2 + R^2\end{aligned}$$

By induction, since $\|\theta_1\|^2 = 0$, it follows that:

$$\|\theta_{k+1}\|^2 \leq kR^2 \tag{5}$$

3. Conclusion

Combining the squared version of (4) with (5):

$$k^2\gamma^2 < \|\theta_{k+1}\|^2 \leq kR^2$$

This simplifies to $k^2\gamma^2 < kR^2$, which implies:

$$k < \frac{R^2}{\gamma^2}$$

The number of updates k is upper-bounded, proving the algorithm must terminate. □

2.2 Greatest Common Divison problem

Algorithm 2 Recursive GCD

```
1: procedure GCD(x, y)
2:     r ← y (mod x)
3:     if r = 0 then
4:         return x
5:     else
6:         return GCD(r, x)
7:     end if
8: end procedure
```

Proof Logic: Based on the Euclidean Lemma: $\gcd(y, x) = \gcd(x, y \pmod{x})$. We reduce the problem size in each step until we hit the base case $\gcd(d, 0) = d$.

Proof Method: Strong Induction on the value of x.

Proof:

- **Base Case:** If $y \pmod{x} = 0$, the algorithm returns x. Since x divides y and x is the largest divisor of itself, $\gcd(x, y) = x$. Correct.
- **Inductive Step:** Assume $GCD(r, x)$ correctly returns $\gcd(r, x)$ for all $r < x$.
- By the Euclidean Lemma, $\gcd(y, x) = \gcd(x, r)$ where $r = y \pmod{x}$.
- Since $r < x$, the recursive call $\text{GCD}(r, x)$ is guaranteed to return $\gcd(r, x)$ by the inductive hypothesis.
- Therefore, Result $= \gcd(x, r) = \gcd(y, x)$. □

Algorithm 3 Iterative GCD

```
1: procedure GCD(x, y)
2:     a ← x, b ← y
3:     while b ≠ 0 do
4:         r ← a (mod b)
5:         a ← b
6:         b ← r
7:     end while
8:     return a
9: end procedure
```

Proof Logic: Maintain a relationship between variables a and b such that their GCD never changes, even as the numbers themselves get smaller.

Proof Method: Loop Invariant.

Proof: Let Invariant $I : \gcd(a, b) = \gcd(x, y)$.

- **Initialization:** Initially $a = x, b = y$, so $\gcd(x, y) = \gcd(x, y)$. I is true.

- **Maintenance:** Let a_{old}, b_{old} be values before a loop. After lines 4-6: $a_{new} = b_{old}$ and $b_{new} = a_{old} \pmod{b_{old}}$. By Euclidean Lemma, $\gcd(a_{old}, b_{old}) = \gcd(b_{old}, a_{old} \pmod{b_{old}})$. So $\gcd(a_{new}, b_{new}) = \gcd(a_{old}, b_{old}) = \gcd(x, y)$. I is preserved.

- **Termination:** Loop ends when $b = 0$. By I, $\gcd(a, 0) = \gcd(x, y)$. Since $\gcd(a, 0) = a$, the algorithm returns the correct GCD. □